

HELLO

I'M NATHAN HERALD

I'M @myobie ON THE INTERNET

I WORKED ON WUNDERLIST

**AUTOCORRECT TRIES
TO CORRECT**

WUNDERLIST TO WANDERLUST

**SO IF YOU SEE THAT, THEN THAT
MEANS I LOST THE BATTLE**

I NOW WORK FOR MICROSOFT



**I WILL LEAVE A LOT OF TIME
FOR QUESTIONS**

SO PLEASE THINK OF SOME

THERE ARE NO BAD QUESTIONS

**BTW, I NORMALLY USE MY
OWN PRESENTATION
SOFTWARE**

**DON'T DO THIS UNLESS YOU
WANT TO BE SUPER STRESSED**



**BUT I'M IN THE MIDDLE OF
A REFACTORING...**





SO KEYNOTE...

**ALSO, JUST FYI,
I LOVE THIS
LOGITECH SPOTLIGHT REMOTE**



PERSISTING STATE

IN A DISTRIBUTED SYSTEM

**IN AN OBJECT ORIENTED
SYSTEM**

“OFFLINE FIRST”

“I would rather have a system that's 75% 'down' but users are fine, than a system 99.99% 'up' but user experience is impacted.” #strangeloop

–@mipsytipsy

USING A FUNCTIONAL LANGUAGE*

#BUZZ WORDS

#THOUGHTLEADERING



[FLICKR.COM/PHOTOS/ODDSOCK](https://www.flickr.com/photos/oddssock)

**WHAT IF EVERY USER HAD
THEIR OWN DATABASE?**

SYSTEM

PHONE

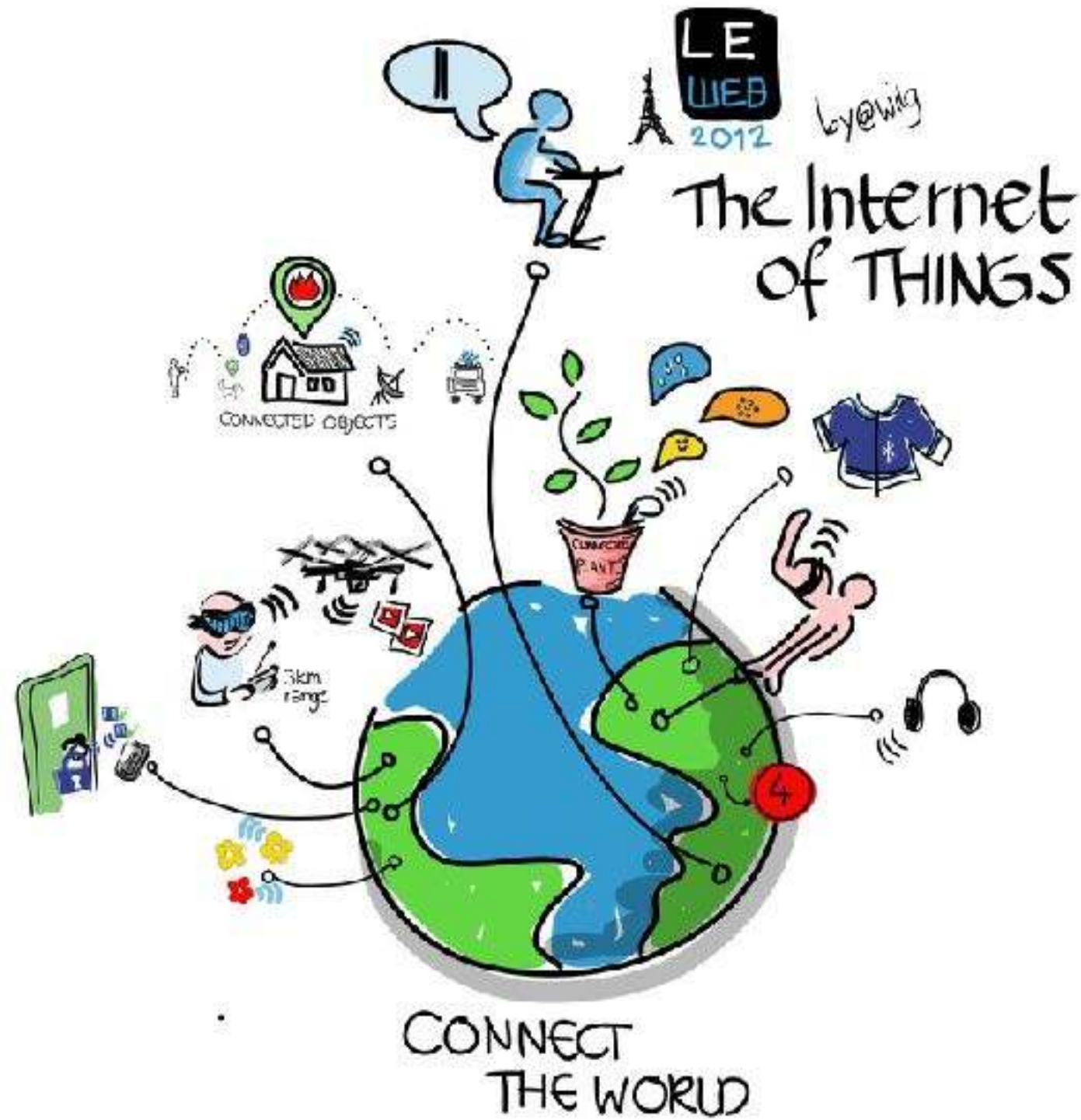
APP

NETWORK

LOAD BALANCER

SERVER

STORAGE



[FLICKR.COM/PHOTOS/WILGENGEBROED](https://www.flickr.com/photos/wilgengebroed)



EVERYTHING IS A THING

NAME THEM

TASKS

LISTS

REMINDERS

NOTES

**THIS IS HOW
WUNDERLIST WORKS**

COLLECTIONS ARE THINGS

**SERIALIZED ACCESS USING
DB TRANSACTIONS***

APP ↔ DB

APP ↔ DB

APP ↔ DB

APP ↔ DB

APP ↔ DB

TASKS IS ACTUALLY

TASKS-PERMISSIONS

TASKS-READER

TASKS-WRITER

**APIS ARE DESIGNED TO
SEEM LIKE DOWNLOADING
DATABASE TABLES**

**ALL UI RELATED QUERIES ARE
DONE AGAINST A LOCAL
DEVICE STORAGE**

**A BACKGROUND PROCESS
SYNCS DATA INTO THE LOCAL
STORAGE AND NOTIFIES THE UI
THERE ARE UPDATES AVAILABLE**

**MSFT SERVICE FABRIC OFFERS
“STATEFUL COLLECTIONS”
WHICH ARE SIMILAR FOR THE
SERVER PART**

**POUCHDB IS AN EXAMPLE OF
SYNCING DATA LOCALLY AND
THEN USING LOCAL QUERIES
TO RUN A UI**

[Blog](#)[Guides](#)[API](#)[Learn](#)[Download v6.3.4](#)[GitHub](#)

The Database that Syncs!

PouchDB is an open-source JavaScript database inspired by [Apache CouchDB](#) that is designed to run well within the browser.

PouchDB was created to help web developers build applications that work as well offline as they do online.

It enables applications to store data locally while offline, then synchronize it with CouchDB and compatible servers when the application is back online, keeping the user's data in sync no matter where they next login.

[Learn more](#)

```
var db = new PouchDB('dbname');

db.put({
  _id: 'dave@gmail.com',
  name: 'David',
  age: 69
});

db.changes().on('change', function() {
  console.log('Ch-Ch-Changes');
});

db.replicate.to('http://example.com/mydb');
```



Cross Browser

Works in Firefox, Chrome, Opera, Safari, IE and Node.js



Lightweight

PouchDB is just a script tag and 46KB (gzipped) away in the browser, or `$ npm`



Easy to Learn

Requires some programming knowledge, however PouchDB is a piece



Open Source

Everything is developed out in the open on GitHub, contributors always

oo?

To Do

And other "to do" list

TASK 4

LIST 38

USER 42

HOW?

AKKA

[TRY AKKA](#)[DOCUMENTATION](#)[BLOG](#)[GET INVOLVED](#)

Build powerful reactive, concurrent, and distributed applications more easily

Akka is a toolkit for building highly concurrent, distributed, and resilient message-driven applications for Java and Scala

[TRY AKKA](#)

Akka is *the* implementation of the [Actor Model](#) on the JVM.



Simpler Concurrent & Distributed Systems

[Actors](#) and [Streams](#) let you build systems that scale *up*, using the resources of a server more efficiently, and *out*, using multiple servers.



Resilient by Design

Building on the principles of [The Reactive Manifesto](#) Akka allows you to write systems that self-heal and stay responsive in the face of failures.



High Performance

Up to 50 million msg/sec on a single machine. Small memory footprint; ~2.5 million actors per GB of heap.



Elastic & Decentralized

Distributed systems without single points of failure. Load balancing and adaptive routing across nodes. [Event Sourcing](#) and [CQRS](#) with [Cluster Sharding](#). [Distributed Data](#) for eventual consistency using CRDTs.



Reactive Streaming Data

Asynchronous non blocking stream processing with backpressure. Fully async and streaming [HTTP server and client](#) provides a great platform for building microservices. Streaming integrations with [Alpakka](#)

AKKA & WEBSOCKETS

**SOME OBJECTS ARE
MORE MONOLITHIC**

THIS IS FINE™



"ON FIRE" BY K.C. GREEN

BEHAVIOUR

BEHAVIOUR

FUNCTIONS

OBJECT STATE

TASK TITLE

LIST TITLE

CLOCKS

en.wikipedia.org

Not logged in | [Talk](#) | [Contributions](#) | [Create account](#) | [Log in](#)

Article

Talk

Read

Edit

View history

Search Wikipedia



WIKIPEDIA

The Free Encyclopedia

Main page

Contents

Featured content

Current events

Random article

Donate to Wikipedia

Wikipedia store

Interaction

Help

About Wikipedia

Community portal

Recent changes

Contact page

Tools

What links here

Related changes

Upload file

Special pages

Permanent link

Page information

Wikidata item

Cite this page

Print/export

Create a book

Download as PDF

Printable version

Languages



WIKIPEDIA
ASIAN MONTH

This November is the Wikipedia Asian Month. [Come join us.](#)

Lamport timestamps

From Wikipedia, the free encyclopedia

The algorithm of **Lamport timestamps** is a simple algorithm used to determine the order of events in a [distributed computer system](#). As different nodes or processes will typically not be perfectly synchronized, this algorithm is used to provide a [partial ordering](#) of events with minimal overhead, and conceptually provide a starting point for the more advanced [vector clock](#) method. They are named after their creator, [Leslie Lamport](#).

Distributed algorithms such as resource synchronization often depend on some method of ordering events to function. For example, consider a system with two processes and a disk. The processes send messages to each other, and also send messages to the disk requesting access. The disk grants access in the order the messages were sent. For example process A sends a message to the disk requesting write access, and then sends a read instruction message to process B. Process B receives the message, and as a result sends its own read request message to the disk. If there is a timing delay causing the disk to receive both messages at the same time, it can determine which message [happened-before](#) the other: (*A happens-before B* if one can get from *A* to *B* by a sequence of moves of two types: moving forward while remaining in the same process, and following a message from its sending to its reception.) A logical clock algorithm provides a mechanism to determine facts about the order of such events.

Lamport invented a simple mechanism by which the [happened-before](#) ordering can be captured numerically. A Lamport logical clock is an incrementing software counter maintained in each process.

Conceptually, this logical clock can be thought of as a clock that only has meaning in relation to messages moving between processes. When a process receives a message, it resynchronizes its logical clock with that sender. The above mentioned vector clock is a generalization of the idea into the context of an arbitrary number of parallel, independent processes.

Contents [\[hide\]](#)

1 Algorithm

2 Considerations

3 Causal Ordering

4 Implications

5 Lamport's logical clock in distributed systems

OWNERSHIP

MEMBERSHIP

DEVICE SUB-SET

WHERE DO WE SAVE STUFF?

**IF YOU DON'T KNOW WHAT TO
DO, USE POSTGRES**

**IF YOU DON'T KNOW WHAT TO
DO ON DEVICE, USE SQLITE**



**WHAT DOES THIS HAVE
TO DO WITH ANYTHING?
I KNOW ALL THIS.**

SURE. SORRY.

**WUNDERLIST WAS DESIGNED
AND BUILT AS AN OBJECT
ORIENTED DISTRIBUTED SYSTEM**

OS PROCESSES

RUNIT

CONSUL

**LAUNCH A WHOLE VM
FROM AN IMAGE TO RUN
ONE PROCESS**

**LAUNCH A WHOLE VM
FROM AN IMAGE TO RUN
ONE PROCESS**

SERIOUSLY.

**LAUNCH A NEW PROCESS
ANYTIME, ANYWHERE**

LISTENS ON :8000

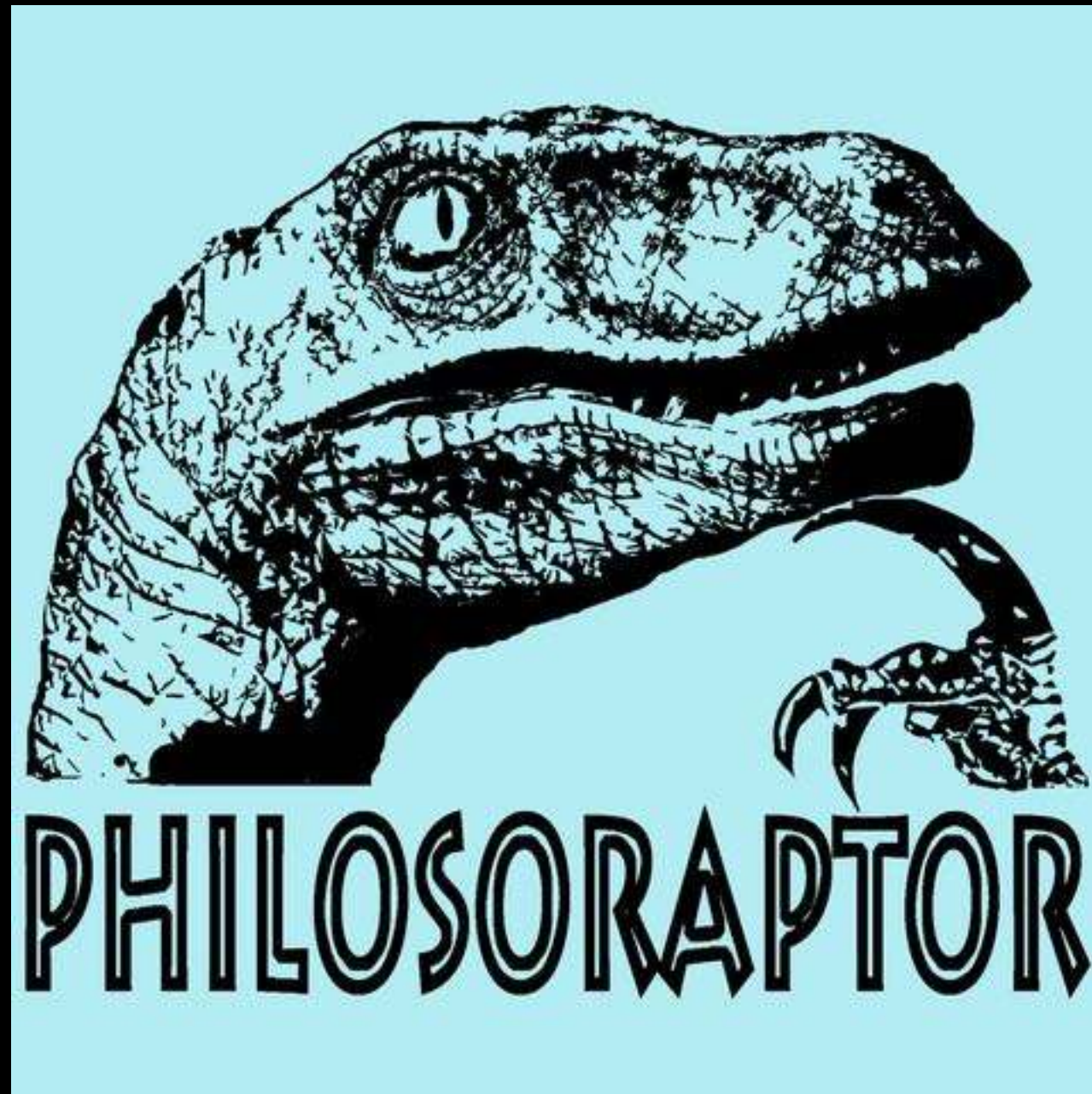
SPEAKS HTTP

SERIALIZES WITH JSON

STATSD

**I CALL THIS OUR
“CRAPPY ACTOR
ARCHITECTURE”**

CRAPTOR?



[LONELYDINOSAUR.COM/APPAREL/PHILOSORAPTOR.HTML](http://lonelydinosaur.com/apparel/philosoraptor.html)

ERLANG

SYSTEM

CLUSTER, NODES, RPC

OBJECTS

PROCESSES

STATE

UH, state

WHERE TO SAVE IT?

POSTGRES

DOCUMENT?

BLOB?

**WHAT IF EVERY USER HAD
THEIR OWN DATABASE?**

OK

ACTORS

**HOW DO I KNOW I ONLY HAVE
ONE ACTOR FOR A PERSON AT
A TIME?**

AKKA

[TRY AKKA](#)[DOCUMENTATION](#)[BLOG](#)[GET INVOLVED](#)

Build powerful reactive, concurrent, and distributed applications more easily

Akka is a toolkit for building highly concurrent, distributed, and resilient message-driven applications for Java and Scala

[TRY AKKA](#)

Akka is *the* implementation of the [Actor Model](#) on the JVM.



Simpler Concurrent & Distributed Systems

[Actors](#) and [Streams](#) let you build systems that scale *up*, using the resources of a server more efficiently, and *out*, using multiple servers.



Resilient by Design

Building on the principles of [The Reactive Manifesto](#) Akka allows you to write systems that self-heal and stay responsive in the face of failures.



High Performance

Up to 50 million msg/sec on a single machine. Small memory footprint; ~2.5 million actors per GB of heap.



Elastic & Decentralized

Distributed systems without single points of failure. Load balancing and adaptive routing across nodes. [Event Sourcing](#) and [CQRS](#) with [Cluster Sharding](#). [Distributed Data](#) for eventual consistency using CRDTs.



Reactive Streaming Data

Asynchronous non blocking stream processing with backpressure. Fully async and streaming [HTTP server and client](#) provides a great platform for building microservices. Streaming integrations with [Alpakka](#)

AKKA

I LOVE AKKA

AKKA

DISTRIBUTED ACTOR REGISTRY



```
1 class SimpleClusterListener extends Actor with ActorLogging {
2
3   val cluster = Cluster(context.system)
4
5   override def preStart(): Unit = {
6     cluster.subscribe(self, initialStateMode = InitialStateAsEvents,
7       classOf[MemberEvent], classOf[UnreachableMember])
8   }
9   override def postStop(): Unit = cluster.unsubscribe(self)
10
11   def receive = {
12     case MemberUp(member) =>
13       log.info("Member is Up: {}", member.address)
14     case UnreachableMember(member) =>
15       log.info("Member detected as unreachable: {}", member)
16     case MemberRemoved(member, previousStatus) =>
17       log.info(
18         "Member is Removed: {} after {}",
19         member.address, previousStatus)
20     case _: MemberEvent => // ignore
21   }
22 }
```

AKKA

I DON'T LOVE SCALA

AKKA

OR JAVA

AKKA

**SOME DO. AND THIS
IS A GREAT CHOICE.**



VIMEO.COM/214024434

ORLEANS

ORLEANS

**INCLUDES A DISTRIBUTED
PROCESS REGISTRY**

ORLEANS

USES A DHT

ORLEANS

**AUTO-SPAWN A “GRAIN”
ANYWHERE**



```
1 type Grain1() =  
2     inherit Orleans.Grain()  
3  
4     interface IHello with  
5  
6         override this.SayHello(greeting:string) =  
7             Task.FromResult<string>("This comes from F#!")
```

ORLEANS

**THE DHT IS EVENTUALLY
CONSISTENT THROUGHOUT
THE CLUSTER**

ORLEANS

**CONSISTENCY OF THE
PERSISTED DATA IS
GUARANTEED BY
THE BACKEND STORAGE**

**CONSISTENCY OF THE
PERSISTED DATA IS
GUARANTEED BY
THE BACKEND STORAGE**

THE DEFAULT IS AZURE TABLES

**CONSISTENCY OF THE
PERSISTED DATA IS
GUARANTEED BY
THE BACKEND STORAGE**

ETAGS WHEN WRITING

**CONSISTENCY OF THE
PERSISTED DATA IS
GUARANTEED BY
THE BACKEND STORAGE**

**ETAG MISMATCH MEANS THIS
ACTOR HAS OLD STATE,
SO SELF-TERMINATE**

**CONSISTENCY OF THE
PERSISTED DATA IS
GUARANTEED BY
THE BACKEND STORAGE**

**THIS MEANS READS
CAN BE STALE**

**ONE CAN USE CLOCKS OR
SOME OTHER SCHEME TO MAKE
READING STALE DATA SAFE**

ERLANG

ERLANG

**IN SOME WAYS, ERLANG IS THE
ONLY TRULY OBJECT ORIENTED
LANGUAGE**

ERLANG

**IN SOME WAYS, ERLANG IS THE
ONLY TRULY OBJECT ORIENTED
LANGUAGE WITHOUT NEEDING
ADDITIONAL LIBRARIES**

ERLANG

ALSO, IT'S FUNCTIONAL

ERLANG

ELIXIR IS MY FAVORITE LISP

ERLANG

**WE CAN ARGUE ABOUT
ALL THIS LATER**

ERLANG

global

ERLANG

pg2

```
1 defmodule Phoenix.PubSub.PG2 do
2   use Supervisor
3   def start_link(name, opts) do
4     supervisor_name = Module.concat(name, Supervisor)
5     Supervisor.start_link(__MODULE__, [name, opts], name: supervisor_name)
6   end
7
8   def init([server, opts]) do
9     scheduler_count = :erlang.system_info(:schedulers)
10    pool_size = Keyword.get(opts, :pool_size, scheduler_count)
11    node_name = opts[:node_name]
12    dispatch_rules = [{:broadcast, Phoenix.PubSub.PG2Server, [opts[:fastlane], server, pool_size]},
13                      {:direct_broadcast, Phoenix.PubSub.PG2Server, [opts[:fastlane], server, pool_size]},
14                      {:node_name, __MODULE__, [node_name]}]
15
16    children = [
17      supervisor(Phoenix.PubSub.LocalSupervisor, [server, pool_size, dispatch_rules]),
18      worker(Phoenix.PubSub.PG2Server, [server, pool_size]),
19    ]
20
21    supervise children, strategy: :rest_for_one
22  end
23
24  def node_name(nil), do: node()
25  def node_name(configured_name), do: configured_name
26 end
```

```
1 defmodule Phoenix.PubSub.PG2Server do
2   @moduledoc false
3
4   use GenServer
5   alias Phoenix.PubSub.Local
6
7   def start_link(server_name, pool_size) do
8     GenServer.start_link __MODULE__, {server_name, pool_size}, name: server_name
9   end
10
11   def init({server_name, pool_size}) do
12     pg2_group = pg2_namespace(server_name)
13     :ok = :pg2.create(pg2_group)
14     :ok = :pg2.join(pg2_group, self())
15
16     {:ok, %{name: server_name, pool_size: pool_size}}
17   end
18
19   def handle_info({:forward_to_local, fastlane, from_pid, topic, msg}, state) do
20     # The whole broadcast will happen inside the current process
21     # but only for messages coming from the distributed system.
22     Local.broadcast(fastlane, state.name, state.pool_size, from_pid, topic, msg)
23     {:noreply, state}
24   end
25
26   defp get_members(server_name) do
27     :pg2.get_members(pg2_namespace(server_name))
28   end
29
30   defp get_members(server_name, node_name) do
31     [{server_name, node_name}]
32   end
33
34   defp pg2_namespace(server_name), do: {:phx, server_name}
35 end
```


ERLANG

**ALL GOOD ERLANG
PROCESS REGISTRIES
THAT I KNOW OF ARE
EVENTUALLY CONSISTENT**

ERLANG

STATE READS CAN BE STALE

ERLANG

**BECAUSE YOU MIGHT ASK AN
OUT OF DATE PROCESS**

ERLANG

WHAT ABOUT Elixir.Registry?

ERLANG

WHAT ABOUT Elixir.Registry?

SINGLE NODE.

ERLANG

**CAN EASILY DO THE SAME AS
ORLEANS AND LET THE
STORAGE LAYER BLOW UP ON
STALE WRITES**

ERLANG

erLeans

ERLANG

erleans

**GITHUB.COM/SPACETIME-IOT/
ERLEANS**

ERLANG

**BUT HONESTLY, I JUST WANT
CONSISTENT READS AND
WRITES**

ERLANG

**I WANT TO WRITE CODE THAT
WORKS ON A SINGLE NODE
AND THEN HAVE IT WORK THE
SAME WHEN DISTRIBUTED**

ERLANG

**GITHUB.COM/MYOBIE/
GEN_PERSISTED_SERVER**



```
1  defmodule Example do
2    use GenServer
3
4    def init(state) when state == %{}, do: {:ok, %{ "calls" => 0 }}
5    def init(state), do: {:ok, state}
6
7    defp increment(state),
8      do: Map.update(state, "calls", 0, fn count -> count + 1 end)
9
10   def handle_call(:hello, _from, state) do
11     {:reply, :goodbye, increment(state)}
12   end
13
14   def handle_call(:count, _from, state) do
15     {:reply, Map.get(state, "calls"), state}
16   end
17 end
18
```



```
1  defmodule Example do
2    use GenPersistedServer, adapter: Adapter
3
4    def init(state) when state == %{}, do: {:ok, %{ "calls" => 0 }}
5    def init(state), do: {:ok, state}
6
7    defp increment(state),
8      do: Map.update(state, "calls", 0, fn count -> count + 1 end)
9
10   def handle_call(:hello, _from, state) do
11     {:reply, :goodbye, increment(state)}
12   end
13
14   def handle_call(:count, _from, state) do
15     {:reply, Map.get(state, "calls"), state}
16   end
17 end
18
```

ERLANG

**GITHUB.COM/MYOBIE/
ZK_REGISTRY**



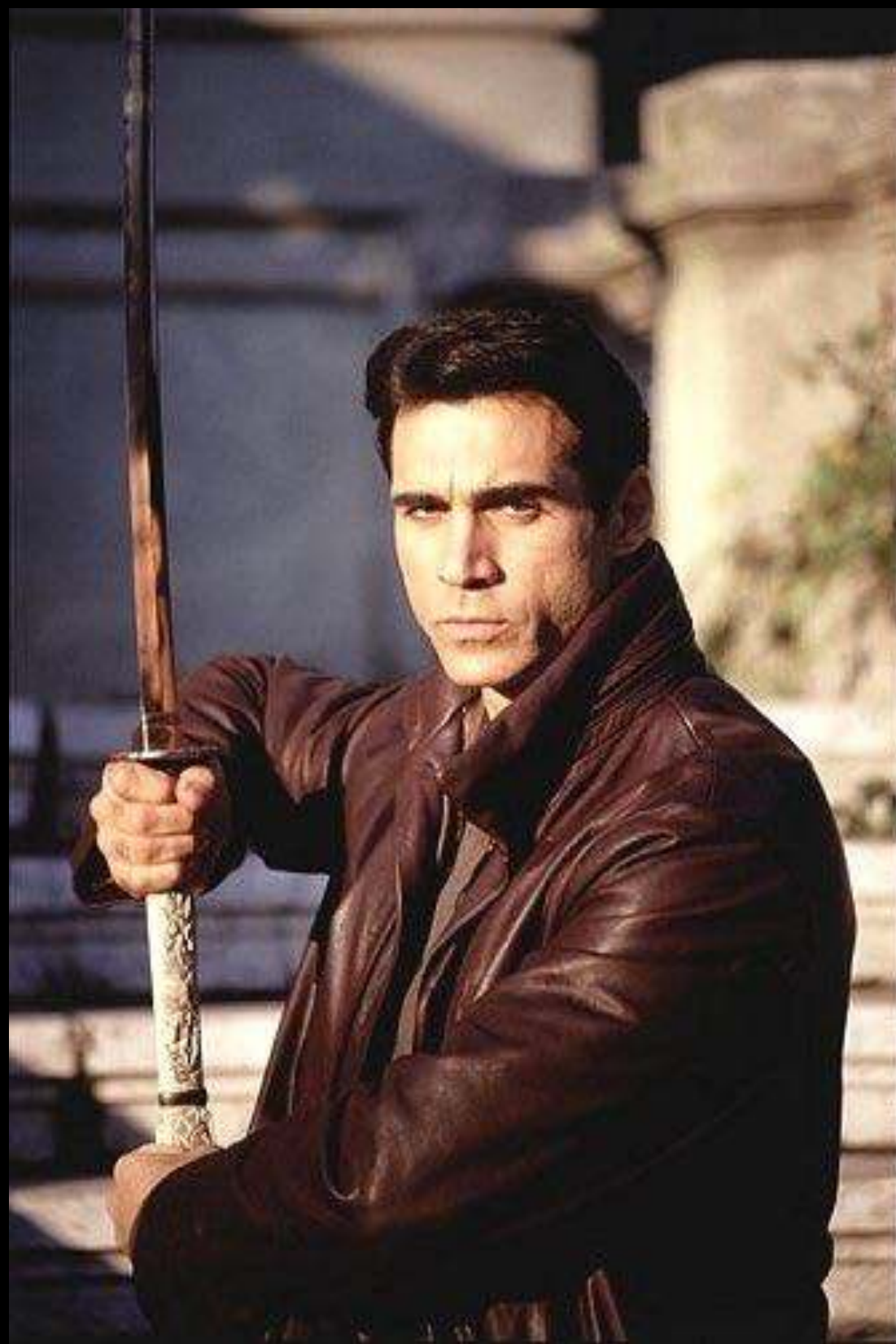
```
1 GenServer.start_link(SomeServer, [])
```



```
1 GenServer.start_link(SomeServer,  
2   [],  
3   name: { :via, ZKRegistry, { :task, "123" } })
```


ERLANG

**GITHUB.COM/MYOBIE/
HIGHLANDER**



**SIMILAR TO MY PRESENTATION
SOFTWARE, THESE ARE
UNFINISHED.**

**SIMILAR TO MY PRESENTATION
SOFTWARE, THESE ARE
UNFINISHED**

THERE IS A PATTERN...

OTHER THINGS TO LOOK AT

**GITHUB.COM/LASP-LANG/
LASP_PG**

VIMEO.COM/191485103

VIMEO.COM/191485103

**DECLARATIVE, SECURE,
CONVERGENT EDGE
COMPUTATION:
AN APPROACH FOR THE
INTERNET OF THINGS**

PROTO.ACTOR

[HOME](#)[TEAM](#)[BLOG](#)[DOCUMENTATION](#)[CODE](#)[PROJECT CHAT](#)

proto

actor

Cross-platform actors for .NET, Go, Java and Kotlin



Simple Concurrency & Distribution

Asynchronous and Distributed
by design. High level.



Extreme Performance

Capable of more than two
million messages per second
between processes.



Resilient by Design

Write systems that self-heal
using supervisor hierarchies.



Built on standards

ProtoAct is built on top of
Google's gRPC and Protobuf,
taking advantage of proven



“I don’t do erlang”

–Maybe You

**THAT'S ALRIGHT.
I FORGIVE YOU.**

**WUNDERLIST DOESN'T
USE ERLANG EITHER**

**WE ARCHITECTED OUR
INFRASTRUCTURE WITH
OO CONCEPTS**

**WE ARCHITECTED OUR
INFRASTRUCTURE WITH
OO CONCEPTS**

**WHAT ARE THE OBJECTS AND
HOW DO THEY COLLABORATE?**

YOU CAN TOO

**IT'S NOT A DOGMA,
IT'S A PHILOSOPHY
TO HELP BREAK UP CONCERNS
AND HELP EVERYONE
UNDERSTAND WHAT
MIGHT BE GOING ON**

EXAMPLE

A GAME

**ORLEANS WAS USED TO
BUILD HALO'S 4 & 5**



HALO 5 IS HEADING TO OVERTIME!

The Overtime content release brings a host of exciting new additions to the Halo 5 experience – from 4K visual enhancements designed to take advantage of the Xbox One X, to the full-fledged return of the fan-favorite Oddball mode, the brand-new Local Server multiplayer feature, and more.

READ MORE

LOCAL SERVER



THE XBOX ONE X IS HERE!



HALO WARS 2 UPDATE



NEW HALO CRATE FIGURE REVEALED

**ORLEANS WAS USED TO
BUILD HALO'S 4 & 5**

**CHANNEL9.MSDN.COM/
EVENTS/BUILD/2014/3-641**

**ORLEANS WAS USED TO
BUILD HALO'S 4 & 5**

GAME

PLAYER

DEVICE

**ORLEANS WAS USED TO
BUILD HALO'S 4 & 5**

SUPER SCALABLE

SUPER PERFORMANT

**GITHUB.COM/SMOLYAKOFF/
CONREIGN**

TIC TAC TOE

TIC TAC TOE

GAME

PLAYER

DEVICE

TIC TAC TOE

GAME

GENSERVER

PLAYER

GENSERVER

DEVICE

IN-MEMORY

TIC TAC TOE

GAME

GENSERVER

PLAYER

GENSERVER

DEVICE

IN-MEMORY

CONNECTION

CHANNEL

TIC TAC TOE

**PERSIST GAME AND PLAYER
STATE TO S3**

TIC TAC TOE

ONLY ONE PROCESS PER GAME

TIC TAC TOE

**ONLY ONE PROCESS PER
PLAYER**

TIC TAC TOE

**ONE PROCESS PER DEVICE
CONNECTION**

TIC TAC TOE

**ONE PROCESS PER DEVICE
CONNECTION**

THANKS PHOENIX

GITHUB.COM/MYOBIE/TTT

I GOT DISTRACTED

[Log In](#)
[Sign Up](#)

[QUESTIONS](#)
[TAGS](#)
[USERS](#)
[BADGES](#)
[UNANSWERED](#)
[ASK QUESTION](#)

Winning strategies in multidimensional tic-tac-toe

↑

9

↓

★

1

This question is a result of having too much free time years ago during military service. One of the many pastimes was playing tic-tac-toe in varying grid sizes and dimensions, and it lead me to a conjecture. Now, after several years of mathematical training at a university, I am still unable to settle the conjecture, so I present it to you.

The classical tic-tac-toe game is played on a 3×3 grid and two players take turns to put their mark somewhere in the grid. The first one to get three collinear marks wins. Collinear includes horizontal, vertical and diagonal lines. Experience shows that the game always ends in a draw if both players play wisely.

Let us write the grid size 3×3 as 3^2 . We can change the edge length by playing on any a^2 grid (where each player tries to get a marks in a row on the $a \times a$ grid). We can also change dimension by playing on any a^d grid, for example $3^3 = 3 \times 3 \times 3$. I want to understand something about this game for general a and d . Let me repeat: The goal is to make a collinear marks.

I assume both players play in an optimal way. It is quite easy to see that the first player wins on a 2^d grid for any $d \geq 2$ but the game is a tie on 2^1 . The game is a tie also on 3^1 and 3^2 , but my experience suggests that the first player wins on 3^3 but the game ties on 4^d for $d \leq 3$. It seems quite credible that if there is a winning strategy on a^d , there is one also on $a^{d'}$ for any $d' \geq d$, since more dimensions to move in gives more room for winning rows. [This answer](#) to a related question tells that for any a there is d so that there is a winning strategy on a^d .

This brings me to the conjecture:

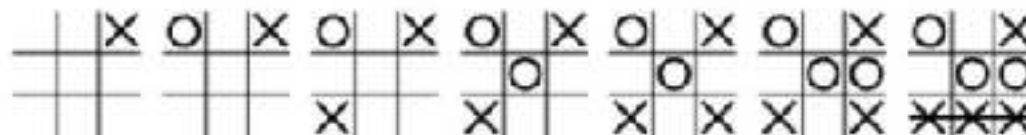
There is a winning strategy for tic-tac-toe on an a^d grid if and only if $d \geq a$. (Refuted by TonyK's answer below.)

PAPERS

A Scientific Study: k-Dimensional Tic-Tac-Toe

Davis Shin

A typical game of Tic-Tac-Toe is played by two people. Each take turn to mark in a three-by-three, two-dimensional grid. The objective of the game is to place three marks of the same kind in a row either in a horizontal, vertical, or diagonal direction. The following picture from Wikipedia demonstrates one of the possible sequences leading to a winning condition in the horizontal direction:



We will expand the popular definition of Tic-Tac-Toe to k-Tic-Tac-Toe, a game of two or more players taking turn to mark in a k-dimensional space, each dimension with a set of domain $Z_k = \{0, 1, 2\}$. Notice we can apply our experience with $Z_0 = \{0, 1\}$. If we define vector addition and scalar multiplication carefully, which we will discuss later, we will have a vector space Z_k^k . The number of grids in a k-dimensional game is g^k . A player wins a game if three marks of the same kind is connected in a line.

0-Tic-Tac-Toe and 1-Tic-Tac-Toe

Since $3^0 = 1$, 0-Tic-Tac-Toe has to be played within one grid, which is not possible.

Similarly, $3^1 = 3$, a player cannot satisfy the winning condition without violating the rules in a 1-Tic-Tac-Toe.

2-Tic-Tac-Toe

This is the original two-dimensional Tic-Tac-Toe. We start our analysis by introducing position vectors that describes the location of a mark, vector addition and scalar multiplication rules in this vector space, and the conditions under which these vectors must meet in order to qualify as a winning set.

The tables to the left illustrate the addition and scalar multiplication rules to maintain the vector space property. These rules are analogous to those of the bit matrices.

.	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

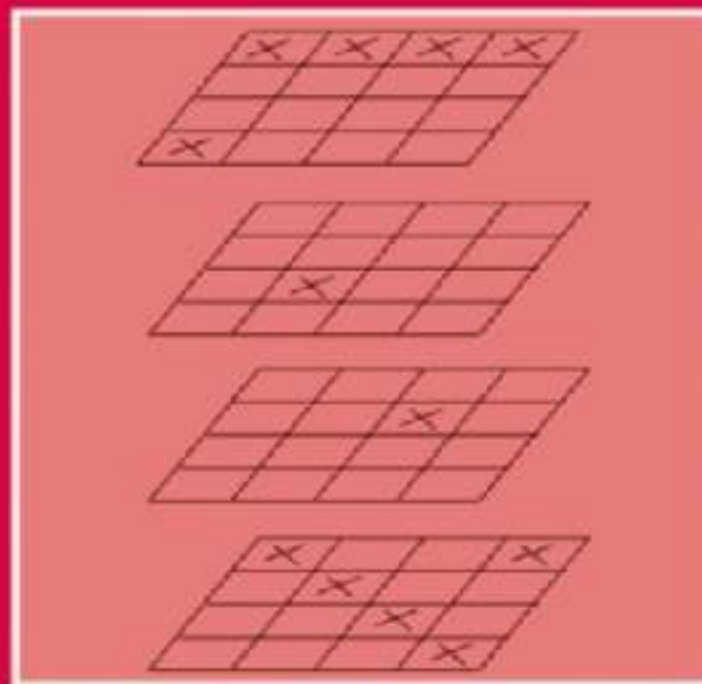
+	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

**MATH.UCDAVIS.EDU/~LINEAR/
STUDENT_CREATIONS/
TICTACTOE.PDF**

COMBINATORIAL GAMES

Tic-Tac-Toe Theory

József Beck



CAMBRIDGE

Combinatorial Games

Traditional game theory has been successful at developing strategy in games of incomplete information: when one player knows something that the other does not. But it has little to say about games of complete information, for example Tic-Tac-Toe, solitaire, and hex. This is the subject of Combinatorial Game Theory. Most board games are a challenge for mathematics: to analyze a position one has to examine the available options, and then the further options available after selecting any option, and so on. This leads to combinatorial chaos, where brute force study is impractical.

In this comprehensive volume, Imre Beck shows readers how to escape from the combinatorial chaos via the fake probabilistic method, a game-theoretic adaptation of the probabilistic method in combinatorics. Using this, the author is able to determine the exact results about infinite classes of many games, leading to the discovery of some striking new duality principles.

IMRE BECK is a Professor in the Mathematics Department of Rutgers University. He has received the Fulkerson Prize for research in Discrete Mathematics and has written around 100 research publications. He is the co-author, with W. L. Chen, of the pioneering monograph *Irregularities of Distribution*.

HELSINKI.FI/~KULIKOV/

MATEM/

BECKCOMBINATORIALGAMES.PDF

ALSO



so

**HOW COULD THINKING ABOUT
PERSISTENCE IN AN OBJECT
ORIENTED WAY CHANGE HOW
YOU BUILD DISTRIBUTED
SYSTEMS?**

**WHAT IF EVERY USER HAD
THEIR OWN DATABASE?**

LET'S DO SOME QUESTIONS

THERE ARE NO BAD QUESTIONS

**BUT, PLEASE DON'T MAKE A
LONG STATEMENT THAT ISN'T A
QUESTION**

**BUT, PLEASE DON'T MAKE A
LONG STATEMENT THAT ISN'T A
QUESTION**

EVERYONE HATES THAT

**OK, I SUPPOSE EVERYONE WILL
PROBABLY CLAP NOW**

**AFTER THAT, THEN LET'S DO
SOME QUESTIONS,
AND THEN EAT LUNCH**

